

The Computational Abilities of Raster Graphics Editors: Running Conway’s Game of Life in Paint

AVA PUN, Carnegie Mellon University, USA

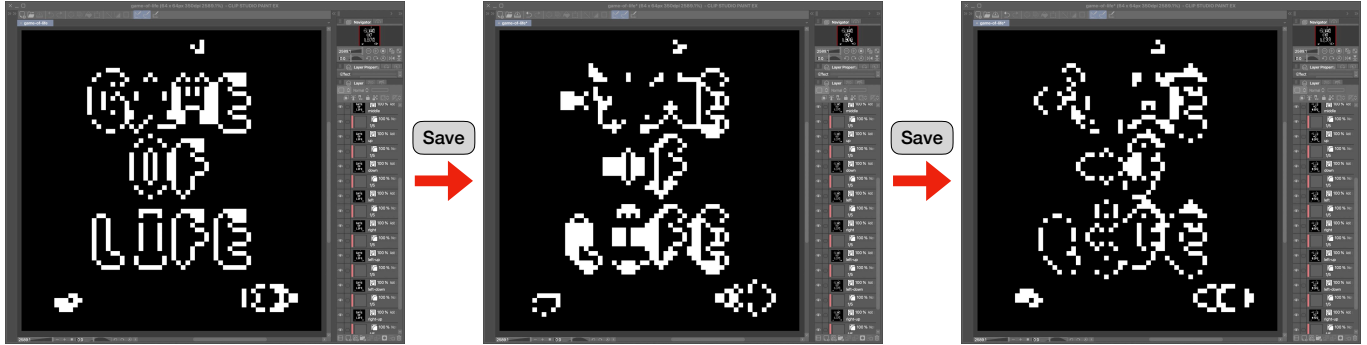


Fig. 1. Conway’s Game of Life, implemented within the raster graphics editor Clip Studio Paint (CSP). The game advances by one timestep when the user hits Save.

Most good raster graphics editors, such as Adobe Photoshop, allow the user to perform mathematical operations on pixel values. These operations can be combined to create programs of surprising complexity. In this paper, I demonstrate how to run one such program, Conway’s Game of Life, in the raster graphics editor Clip Studio Paint (CSP).

Just to be clear, the title is intentionally misleading. We will be using Clip Studio Paint. We will not be using *Microsoft Paint*. Don’t try to do this in Microsoft Paint; it won’t work and you’ll be sad.

This paper is best enjoyed with animated figures. For a blog post containing said figures—and a link to download the CSP file so you can try it out yourself—see <https://avapun.com/blog/game-of-life-csp>. Or just imagine the pictures in this paper are moving. It’s almost the same.

1 Introduction

The intended use of raster graphics editors, such as Adobe Photoshop, is to edit raster graphics [citation needed]. However, many raster graphics editors allow the user to perform mathematical operations on the pixel values in an image. This capability unlocks an entire host of unintended uses.

In this paper, I showcase one such unintended use: running Conway’s Game of Life (fig. 1). By taking advantage of the pixel operations offered by graphics editors, we will implement Conway’s Game of Life in the editor Clip Studio Paint (not to be confused with Microsoft Paint). If you’re impatient and just want to see how it’s done, skip right to section 4. Otherwise, let’s begin with an overview of raster graphics editors.

2 Raster graphics editors

A *raster graphics editor* is a software application that allows users to create and edit *raster images*. Raster images are composed of an array of pixels, as opposed to *vector images*, which are composed of mathematical shapes and curves. Perhaps the most popular raster graphics editor is *Adobe Photoshop* [Adobe 2025], for those are rich

and/or know their way around The Pirate Bay, followed by the open-source *GNU Image Manipulation Program (GIMP)* [GIMP 2025a], for those who run Linux and/or don’t want to give Adobe \$5 billion plus one-seventh of their soul each year. *Clip Studio Paint (CSP)* [Celsys 2025] is a slightly less popular editor, developed by the Japanese company Celsys and known for its comic creation features. Another raster graphics editor is *Microsoft Paint*. Nobody likes Microsoft Paint, not even Microsoft Paint itself.

In this paper, we will be using CSP for reasons detailed in section 5. However, most ideas introduced here also apply to other editors.

2.1 Operations on pixel values

A raster image can be represented using a 2D grid of pixels. Each pixel in the image is associated with a single value, its *brightness*, which is a real number ranging from 0 (black) to 1 (white).¹ In most raster graphics editors,² the user can apply various binary and unary operations to pixel values.

Binary operations. Most good editors³ allow the user to manipulate multiple *layers*, which represent separate raster images in the same editor file. The pixel values of two layers can be combined by setting the *blend mode* of the top layer [Celsys 2024a].⁴ Some useful blend modes include *Add*, which adds the two layers (duh), *Multiply*, which multiplies them (duh), and *Difference*, which divides one by the other. Just kidding. It takes their absolute difference (duh). In our implementation of Conway’s Game of Life, we will only need the *Add* layer (fig. 2).

Note that since pixel values are capped at 1, the *Add* operation has the potential to overflow. If overflow occurs, any overflowing values will be clipped to 1.

¹For simplicity, we shall ignore colours and quantization.

²Except Microsoft Paint.

³And some bad ones, like Microsoft Paint.

⁴But not if you’re using Microsoft Paint.

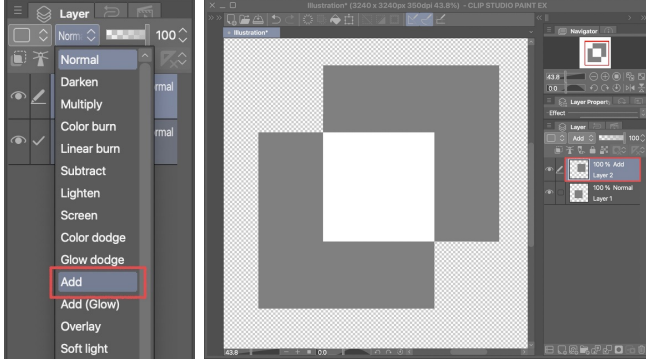


Fig. 2. By selecting the Add layer blend mode (left), the user can add two layers (right) with value 0.5 (50% grey), resulting in a value of 1 (white).

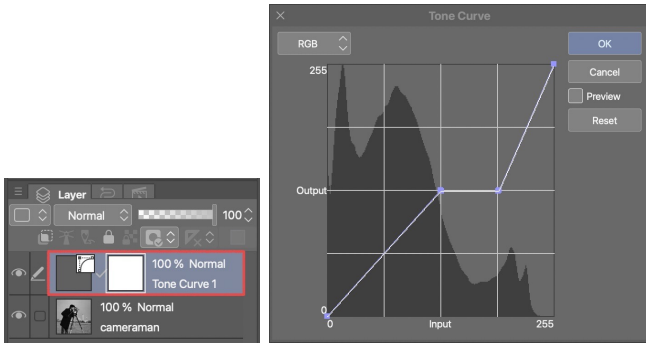


Fig. 3. A tone curve layer (left) that holds a tone curve representing a piecewise linear function (right).

Unary operations. In many editors,⁵ the user can insert a special *tone curve* layer [Celsys 2024c]. This layer allows the user to apply *any continuous function they desire* to the underlying pixel values (fig. 3)! Well, any continuous function that can be drawn using a finite number of control points. If you want Weierstrass your images, you’re out of luck.

By default, a tone curve is applied to the composition of all layers below it. However, the user may apply a *clipping mask* to make a tone curve affect only the layer directly below it.

2.2 Importing files

Some raster graphics editors allow the user to *import* an image file (say A) into another image file (say B) as a separate layer. This process is illustrated in fig. 4. A further subset of editors⁶ support *live updating*: changes to file A will immediately be reflected in file B when file A is saved [Celsys 2024b].

3 Conway’s Game of Life

If you’re enough of a nerd to be reading SIGBOVIK papers, you probably already know what Conway’s Game of Life is. But we’ll give you an overview just in case.

⁵But not Microsoft Paint.

⁶Not including Microsoft Paint.

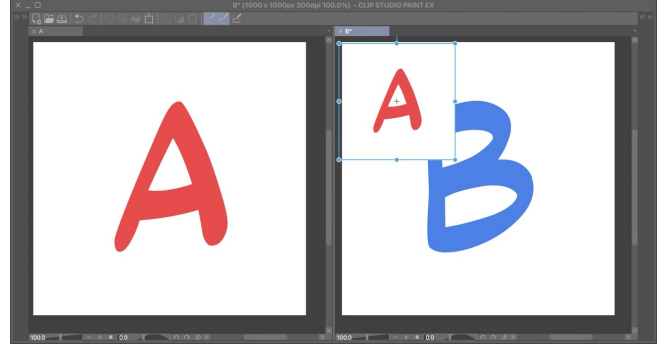


Fig. 4. Importing image file A (left) into file B (right). Image A appears inside B as a separate layer.

Conway’s Game of Life, devised by British mathematician John H. Conway [Gardner 1970], is a game played on a two-dimensional grid. Each cell in the grid can take one of two values: *dead* or *live*. When time moves forward by one step, each cell interacts with its *neighbours*—the eight cells vertically, horizontally, and diagonally adjacent to it—according to the following two rules:

- (1) A live cell stays live if it has exactly two or three live neighbours; otherwise, it dies (from “underpopulation” or “overpopulation”).
- (2) A dead cell becomes live if it has exactly three live neighbours (simulating “reproduction”); otherwise, it stays dead.

We can condense these two rules into one:

- (1) Let a dead cell have a value of 0 and a live cell have a value of 1. A cell’s next value should be 1 if half the cell’s current value, plus the sum of its neighbours’ values, is between 2.5 and 3.5 inclusive. Otherwise, it should be 0.

More formally (because every good paper needs at least one equation), let the set of neighbours of cell p be $N(p)$, and let the value of cell p at time t be v_t^p . Then,

$$v_{t+1}^p = \begin{cases} 1 & \text{if } 2.5 \leq 0.5v_t^p + \sum_{q \in N(p)} v_t^q \leq 3.5 \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

There exist many implementations of Conway’s Game of Life, some of which you can experiment with online [Johnston [n. d.]; Martin [n. d.]]. However, sadly, none of these are implemented in raster graphics editors. We’ll do that in the following section.

4 Running Conway’s Game of Life in CSP

We represent the Game of Life board using a raster image. Each pixel in the image represents a cell; its value is 0 (black) if the cell is dead and 1 (white) if the cell is live. To simulate one step of Conway’s Game of Life, we compute the next pixel values as per eq. (1), using tone curves and the Add layer blend mode (section 2.1).

First, for each pixel p , we wish to add half of its current value to the sum of its neighbours’ values, obtaining $s^p := 0.5v^p + \sum_{q \in N(p)} v^q$. To compute s^p for all pixels, we make eight copies of the game board, each on a separate layer. We shift the copies in the eight cardinal and intercardinal directions. Then, we set their layer blend modes

Algorithm 1: Running one step of Conway’s Game of Life in CSP.

Input: Raster image R composed of black and white pixels, representing Game of Life board; scalar value $\lambda > 3.5$ to prevent overflow during addition

Output: Raster image R' representing new Game of Life board after one timestep

- 1 make eight copies $R_1, \dots, R_8$ of R on separate layers;
- 2 shift $R_1, \dots, R_8$ by 1 in each of the eight cardinal and intercardinal directions;
- 3 apply a tone curve (with clipping mask) to R to scale it by $0.5/\lambda$;
- 4 apply tone curves (with clipping masks) to $R_1, \dots, R_8$ to scale them by $1/\lambda$;
- 5 set the layer blend modes of $R_1, \dots, R_8$ to Add;
- 6 use a tone curve (no clipping mask) to apply

$$f(v) = \begin{cases} 1 & \text{if } 2.5/\lambda \leq v \leq 3.5/\lambda \\ 0 & \text{otherwise} \end{cases}$$

to the entire image composition;

to Add. Finally, we scale the original board by 0.5 using a tone curve. Each pixel p will then contain the sum s^p .

But wait—since pixel values are capped at 1, s^p runs the risk of overflowing. Hence, we shall choose a scalar λ and scale all layers by $1/\lambda$ (again using tone curves) before summing them. Since we need only check if $s^p \in [2.5, 3.5]$, the scalar λ can be any value greater than 3.5.

After scaling and summing, each pixel contains the value s^p/λ . To finish computing eq. (1), we apply a step function (using a tone curve) so that pixel p is 1 if $2.5/\lambda \leq s^p/\lambda \leq 3.5/\lambda$, and 0 otherwise. The full process is summarized in alg. 1.

4.1 Automatically copying the image through file imports

The process we have described is quite tedious. Even if we reuse the tone curve layers across timesteps, we still have to copy the image into eight new layers (and clean up any old copies) during each timestep. Could we automate this process?

Let’s think back to file importing (section 2.2). Recall that if we import image A into image B, then update and save image A, the updated image is *immediately copied* into its file-import layer in B.

Aha! So what we want to do is import our Game of Life image into *itself* eight times. Then, upon saving the file, the updated image will be copied into itself on the eight file-import layers. But is it even possible to import a file into itself? In CSP, yes it is (fig. 5)!

By using file imports to copy the image in line 1 of alg. 1, we need only perform lines 1–6 once. When we press Save, CSP automatically runs lines 1–2 again as it updates the file-import layers, and the tone curves and layer blend modes from lines 3–6 are reused, hence performing one step of the Game of Life (fig. 1). As a bonus, we do not need to clean up any old copies of the image, since the content of the file-import layers is replaced with each update.

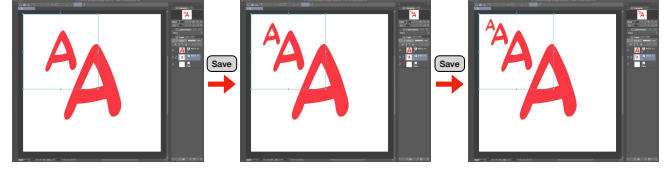


Fig. 5. File A is imported into itself in the top-left corner. When the user hits Save, the entire image is copied into the corner.



Fig. 6. The Custom Filter in Photoshop allows the user to specify a pixel’s new value as a weighted sum of itself and its surrounding pixels.

5 Alternative raster graphics editors

If you’re one of the 99% of people who has never heard of CSP, you might be wondering why I chose to use this relatively obscure editor rather than one like Photoshop. Here are a few reasons:

- (1) I’m more familiar with CSP than any other editor.
- (2) Other editors either lack live-updating file imports, or don’t allow the user to import a file into itself.
- (3) I just wanted to make you all think you were going to read about Microsoft Paint.

Still, although this paper has focused on CSP, it is possible to run Conway’s Game of Life within other raster graphics editors. Let’s see how.

5.1 Adobe Photoshop

Photoshop does not allow the user to import a file into itself. However, it does allow users to record their actions into a *macro*, which can later be replayed with a single click. These macros are called *Actions* [Adobe 2024b]. Instead of using file imports to automate image copying, one can record the steps of alg. 1 into an Action, then play it back to run one step of the Game of Life.

But there exists a simpler solution. Photoshop features a unique operation: the Custom Filter [Adobe 2024a], which lets the user specify a pixel’s new value as a weighted sum of itself and the values of surrounding pixels (fig. 6). We can replace lines 1–5 of alg. 1 with one Custom Filter, and hence run a step of the Game of Life using a single Custom Filter followed by a tone curve. [Egan 2009] has created an Action that does just that, except that it uses a Gradient Map instead of a tone curve to achieve the same effect.

5.2 GNU Image Manipulation Program (GIMP)

To my knowledge, GIMP does not support live-updating file imports, but it does support macros. Unlike Photoshop Actions, these macros take the form of Scheme [GIMP 2025c] or Python [GIMP 2025b] scripts. One could potentially implement alg. 1 using GIMP scripts. But at that point, you’re basically writing the Game of Life in a normal programming language like a normal person, and where’s the fun in that?

5.3 Microsoft Paint

No.

6 Conclusion

We have shown that by combining mathematical operations on pixels, one can run interesting programs like Conway’s Game of Life within the raster graphics editor Clip Studio Paint (CSP). In fact, since Conway’s Game of Life is Turing-complete [Berlekamp et al. 1982], one can even run a Turing machine (or any other program) inside CSP! Creating a working Turing machine within a raster graphics editor is left as an exercise for the reader, where the exercise is “write a paper for next year’s SIGBOVIK” and the reader is me.

References

- Adobe. 2024a. Apply specific filters. <https://helpx.adobe.com/photoshop/using/applying-specific-filters.html>.
- Adobe. 2024b. Creating and recording actions. <https://helpx.adobe.com/photoshop/using/creating-actions.html>.
- Adobe. 2025. Official Adobe Photoshop. <https://www.adobe.com/products/photoshop>.
- Elwyn R. Berlekamp, John H. Conway, and Richard K. Guy. 1982. *Winning Ways for Your Mathematical Plays*. Vol. 2. Academic Press, New York.
- Celsys. 2024a. Blending modes. https://help.clip-studio.com/en-us/manual_en/180_layers/Blending_modes.htm.
- Celsys. 2024b. File objects. https://help.clip-studio.com/en-us/manual_en/180_layers/File_objects.htm.
- Celsys. 2024c. Tonal Correction Effects. https://help.clip-studio.com/en-us/manual_en/390_filters/Tonal_Correction_Effects.htm.
- Celsys. 2025. CLIP STUDIO PAINT. <https://www.clipstudio.net/en>.
- Kelly Egan. 2009. Life From Photoshop. <https://kellyegan.net/2009/life-from-photoshop>.
- Martin Gardner. 1970. The fantastic combinations of John Conway’s new solitaire game “life”. *Scientific American* 223, 4 (1970), 120–123.
- GIMP. 2025a. GIMP - GNU Image Manipulation Program. <https://www.gimp.org>.
- GIMP. 2025b. The “Python-Fu” Submenu. <https://docs.gimp.org/en/gimp-filters-python-fu.html>.
- GIMP. 2025c. Using Script-Fu Scripts. <https://docs.gimp.org/en/gimp-concepts-script-fu.html>.
- Nathaniel Johnston. [n. d.]. Conway’s Game of Life. <https://conwaylife.com>.
- Edwin Martin. [n. d.]. Play John Conway’s Game of Life. <https://playgameoflife.com>.

Received 23 March 2025